

Accessing Your Applications from Outside

Ingress & Egress — Module 6 of 13

Enterprise EKS Platform





Module Agenda

- Why External Access Matters
- NGINX Ingress Controller & IngressClass
- Ingress Resources: Path-Based & Host-Based Routing
- Annotations, TLS Termination & cert-manager
- Gateway API: GatewayClass, Gateway, HTTPRoute
- Traffic Splitting and Canary Deployments
- Enterprise Integration & Egress Controls



Why External Access Matters

Getting Traffic Into Your Cluster

External Access Options

Method	Layer	Use Case	Limitation
NodePort	L4	Dev/testing	Port range 30000-32767, exposed on all nodes
LoadBalancer	L4	Single service exposure	One LB per service (\$\$\$), no path routing
Ingress	L7	HTTP/HTTPS routing	HTTP-only, limited extensibility
Gateway API	L4-L7	Advanced routing	Newer, still maturing ecosystem



NGINX Ingress Controller

Architecture and IngressClass

NGINX Ingress Architecture

How It Works

1. NGINX Ingress Controller pod runs NGINX reverse proxy
2. Exposed via a LoadBalancer Service (single AWS NLB/ALB)
3. Watches for Ingress resources across the cluster
4. Dynamically generates NGINX configuration from Ingress rules
5. Reloads NGINX when Ingress resources change



IngressClass Configuration

```
apiVersion: networking.k8s.io/v1
kind: IngressClass
metadata:
  name: nginx-external
  annotations:
    ingressclass.kubernetes.io/is-default-class: "true"
spec:
  controller: k8s.io/ingress-nginx
---
apiVersion: networking.k8s.io/v1
kind: IngressClass
metadata: { name: nginx-internal }
spec:
  controller: k8s.io/ingress-nginx-internal
```

Recommended Pattern: Run separate IngressClasses for external (internet-facing) and internal (VPC-only) traffic, each with its own controller and load balancer.

Ingress Resources

Path-Based and Host-Based Routing



Path-Based Routing

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: app-ingress
  annotations: { nginx.ingress.kubernetes.io/rewrite-target: /$2 }
spec:
  ingressClassName: nginx-external
  rules:
    - host: app.example.internal
      http:
        paths:
          - path: /api(/|$)(.*)
            pathType: ImplementationSpecific
            backend: { service: { name: api-service, port: { number: 8080 } } }
          - path: /
            pathType: Prefix
            backend: { service: { name: frontend-service, port: { number: 80 } } }
```



Host-Based Routing

```
kind: Ingress
# ...
spec:
  ingressClassName: nginx-external
  rules:
    - host: api.example.internal
      http:
        paths:
          - path: /
            pathType: Prefix
            backend: { service: { name: api-service, port: { number: 8080 } } }
    - host: dashboard.example.internal
      # ... same structure, different backend
  defaultBackend:
    service: { name: default-404-service, port: { number: 80 } }
```

Path Types Explained

pathType	Matching Behavior	Example
Exact	Matches URL path exactly	/api matches /api only, not /api/v1
Prefix	Matches path prefix by / segments	/api matches /api, /api/v1, /api/v1/users
ImplementationSpecific	Depends on IngressClass controller	NGINX supports regex: /api(/ \$)(.*)

Ingress Annotations

Customizing NGINX Behavior



Essential NGINX Ingress Annotations

```
annotations:  
  nginx.ingress.kubernetes.io/ssl-redirect: "true"  
  nginx.ingress.kubernetes.io/proxy-body-size: "50m"  
  nginx.ingress.kubernetes.io/proxy-read-timeout: "120"  
  nginx.ingress.kubernetes.io/limit-rps: "50"  
  nginx.ingress.kubernetes.io/enable-cors: "true"  
  nginx.ingress.kubernetes.io/cors-allow-origin: "https://app.example.com"
```

Advanced Annotations

Session Affinity

```
annotations:  
  nginx.ingress.kubernetes.io/  
    affinity: "cookie"  
  nginx.ingress.kubernetes.io/  
    session-cookie-name: "VSRVSID"  
  nginx.ingress.kubernetes.io/  
    session-cookie-max-age: "172800"
```

Custom Headers & Snippets

```
annotations:  
  nginx.ingress.kubernetes.io/  
    configuration-snippet: |  
      more_set_headers  
        "X-Request-ID: $req_id";  
      more_set_headers  
        "Strict-Transport-Security:  
          max-age=31536000";
```

TLS Termination

Securing Ingress with HTTPS



TLS Termination

Manual Certificate

```
kind: Ingress
# ...
spec:
  ingressClassName: nginx-external
  tls:
  - hosts: [api.example.internal, dashboard.example.internal]
    secretName: app-tls          # kubernetes.io/tls Secret
  rules:
  - host: api.example.internal
    # ... routing rules as before
```

ClusterIssuer (cert-manager)

```
apiVersion: cert-manager.io/v1
kind: ClusterIssuer
metadata: { name: letsencrypt-prod }
spec:
  acme:
    server: https://acme-v02.api.letsencrypt.org/directory
    email: certs@example.com
    privateKeySecretRef: { name: letsencrypt-prod-key }
    solvers:
    - http01:
        ingress: { class: nginx-external }
```

Automated TLS Ingress

```
kind: Ingress
metadata:
  annotations:
    cert-manager.io/cluster-issuer: "letsencrypt-prod" # triggers auto-provisioning
# ...
```

Gateway API

The Evolution of Kubernetes Ingress

Ingress vs Gateway API

Aspect	Ingress	Gateway API
Extensibility	Annotations (untyped strings)	Typed, structured API fields
Role model	Single resource, one owner	Role-oriented (infra, cluster, app)
Protocols	HTTP/HTTPS only	HTTP, TCP, UDP, gRPC, TLS
Traffic splitting	Not native	Built-in weight-based routing
Header matching	Not supported	Native header/query matching
Portability	Vendor-specific annotations	Portable across implementations
Status	Stable, feature-frozen	GA (HTTPRoute), evolving



Gateway API Resource Model



GatewayClass

Role: Infrastructure
Provider

- Defines controller implementation (cluster-scoped)



Gateway

Role: Cluster Operator

- Configures listeners, TLS, and provisions LB



HTTPRoute

Role: Application
Developer

- Defines routing rules; attaches to Gateway

Role Separation: Infrastructure teams manage GatewayClass and Gateway. Application teams create HTTPRoutes. RBAC applies cleanly at each level.

◆ GatewayClass and Gateway

```
# GatewayClass is typically pre-installed by the platform
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata: { name: mycompany-gateway, namespace: gateway-system }
spec:
  gatewayClassName: envoy-gateway
  listeners:
  - name: http
    protocol: HTTP
    port: 80
    allowedRoutes: { namespaces: { from: All } }
  - name: https
    protocol: HTTPS
    port: 443
    tls:
      mode: Terminate
      certificateRefs:
      - { kind: Secret, name: wildcard-tls }
  allowedRoutes:
    namespaces:
      from: Selector
      selector: { matchLabels: { gateway-access: "true" } }
```



HTTPRoute Resource

```
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata: { name: api-routes, namespace: mycompany-apps }
spec:
  parentRefs:
  - { name: mycompany-gateway, namespace: gateway-system, sectionName: https }
  hostnames: ["api.example.internal"]
  rules:
  - matches:
    - path: { type: PathPrefix, value: /v1/dns }
      headers: [{ name: X-API-Version, value: "v1" }]
      backendRefs: [{ name: dns-service-v1, port: 8080 }]
  - matches:
    - path: { type: PathPrefix, value: /v2/dns }
      backendRefs: [{ name: dns-service-v2, port: 8080 }]
```



Traffic Splitting

Canary Deployments with HTTPRoute Weights



Canary Routing with HTTPRoute

Weight-Based

```
kind: HTTPRoute
# ...
spec:
  parentRefs:
  - { name: mycompany-gateway, namespace: gateway-system }
  rules:
  - matches:
    - path: { type: PathPrefix, value: / }
    backendRefs:
    - { name: dns-service-stable, port: 8080, weight: 90 }
    - { name: dns-service-canary, port: 8080, weight: 10 }
```

Header-Based

```
kind: HTTPRoute
# ...
rules:
- matches:
  - headers: [{ name: X-Canary, value: "true" }]
  backendRefs:
  - { name: dns-service-canary, port: 8080 }
- matches:
  - path: { type: PathPrefix, value: / }
  backendRefs:
  - { name: dns-service-stable, port: 8080 }
```



Enterprise Integration

Envoy Gateway + External DNS + cert-manager



Enterprise EKS Ingress Stack



Envoy Gateway

- Gateway API + Envoy data plane
- Advanced L7: gRPC + HTTP



External DNS

- Watches Gateway/Ingress
- Auto-creates Route53 records



cert-manager

- Automated cert lifecycle
- Internal CA, ACME, or Gateway API

Workflow: HTTPRoute → Envoy configures proxy → External DNS creates Route53 record → cert-manager provisions TLS → App is live.

Egress Controls

Managing Outbound Traffic



Why Egress Controls Matter



Security Concerns

- Compromised pods phoning home or exfiltrating data
- Supply chain attacks reaching external registries
- Compliance requirements (PCI-DSS, SOC2)



Control Mechanisms

- **NetworkPolicy egress rules** (Module 8)
- **Egress gateways** (service mesh) or **NAT gateways**
- **Proxy servers** for HTTP traffic

Egress Patterns on EKS

Common Egress Architectures

NAT Gateway (Default EKS)

```
Pod → Node → NAT GW → Internet
                        ↓
                    Fixed Elastic IP
```

- All egress shares NAT GW IP
- Useful for IP allowlisting
- No application-level control

Certificates for External Endpoints

```
# Mount CA bundle for external APIs
volumes:
- name: ca-certs
  secret: { secretName: external-ca-bundle }
# ...
volumeMounts:
- { name: ca-certs, mountPath: /etc/ssl/custom, readOnly: true }
```



Lab 6: Ingress and Gateway API

Hands-On Exercise

- Verify the NGINX Ingress Controller and IngressClass
- Create host-based and path-based Ingress resources
- Configure TLS termination with a self-signed certificate
- Test Ingress annotations: rate limiting, CORS, rewrite
- *Optional:* Deploy Gateway API resources with HTTPRoute traffic splitting
- *Optional:* Configure egress NetworkPolicies to control outbound traffic

Duration: ~30-40 minutes

Key Takeaways

1. Consolidate with Ingress/Gateway — Avoid one LoadBalancer per service. Route through a shared entry point.

2. Gateway API is the future — Typed resources, role separation, and native traffic management replace annotation-driven Ingress.

3. Automate TLS — Use cert-manager for certificate lifecycle. Avoid managing certificates manually.

4. Control egress too — Ingress is only half the story. Egress controls are critical for security and compliance.

? Questions & Discussion

Module 6: Accessing Your Applications from Outside — Ingress
& Egress

Up Next: Module 7 — RBAC & Security